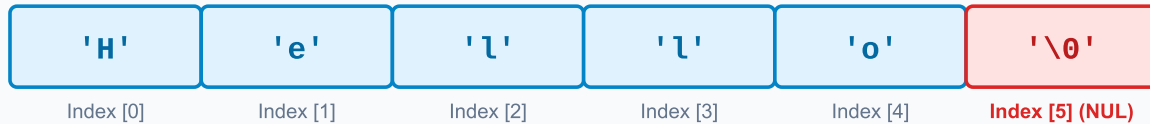


# String Foundations & LeetCode Problem Bank

A rigorous handbook covering memory layout, null termination, pointer mechanics, and 15 curated LeetCode algorithmic solutions in pure C.

## 1. Fundamental C String Architecture

In C, a string is not a distinct primitive data type. Instead, a string is defined as a contiguous one-dimensional array of `char` values terminated by a special character called the **null terminator** (`'\0'`, ASCII value `0`). Without this terminator, standard library functions cannot determine where string data ends in memory, resulting in undefined behavior or memory buffer overflows.



## 2. Declaration, Pointers & Critical Memory Differences

### Array vs Pointer Literals

`char a[] = "Hello";` creates a mutable local copy on the stack (size 6 bytes). `char *p = "Hello";` points directly to string literal memory stored in read-only section (`.rodata`). Modifying `p[0] = 'M';` triggers an immediate segmentation fault.

### Size vs Length Discrepancy

`sizeof()` reports total allocated memory in bytes, including `'\0'`. `strlen()` iterates over characters until encountering `'\0'`, returning character count without the terminator. Always allocate `malloc(len + 1)` for dynamic strings.

### Standard String Operations

Found in `<string.h>`: `strlen(s)`, `strcpy(dest, src)`, `strcmp(s1, s2)`, and `strcat(dest, src)`. In competitive programming, re-implementing these inline avoids redundant traversal overhead.

### LeetCode Return Conventions

When LeetCode requires returning a new string (e.g. `char*`), allocate memory dynamically with `malloc` and explicitly append `ret[len] = '\0';`. Never return the address of a local stack array.

### Key Rule for Freshmen Coding in C:

Always remember that arrays degrade to pointers when passed into functions. When writing algorithms on strings, either track the length explicitly or check for `str[i] != '\0'` in loops.

### 3. Fundamental Problems (LeetCode Easy) – Part I

#### Problem 1: Reverse String (LeetCode 344)

Easy

**Task:** Write a function that reverses an array of characters in-place with  $O(1)$  extra memory.

```
void reverseString(char* s, int sSize) {
    int left = 0, right = sSize - 1;
    while (left < right) {
        char temp = s[left];
        s[left] = s[right];
        s[right] = temp;
        left++;
        right--;
    }
}
```

**Core Concept:** Classic Two-Pointer technique. Swapping items from boundaries moving inward avoids buffer allocation and yields  $O(N)$  runtime.

#### Problem 2: To Lower Case (LeetCode 709)

Easy

**Task:** Convert uppercase letters in string `s` to lowercase and return the modified string.

```
char* toLowerCase(char* s) {
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] >= 'A' && s[i] <= 'Z') {
            s[i] = s[i] + ('a' - 'A'); // ASCII delta: +32
        }
    }
    return s;
}
```

**Core Concept:** Direct ASCII manipulation. In ASCII, uppercase 'A' is 65 and lowercase 'a' is 97. Modifying in-place preserves memory.

#### Problem 3: Length of Last Word (LeetCode 58)

Easy

**Task:** Given a string of words and spaces, return the length of the last word in the string.

```
int lengthOfLastWord(char* s) {
    int i = strlen(s) - 1, len = 0;
    while (i >= 0 && s[i] == ' ') i--; // Skip trailing spaces
    while (i >= 0 && s[i] != ' ') {
        len++;
        i--;
    }
    return len;
}
```

**Core Concept:** Reverse linear scan. By searching backwards from the end of string, you avoid parsing irrelevant preceding words.

## 4. Fundamental Problems (LeetCode Easy) – Part II

### Problem 4: Valid Palindrome (LeetCode 125)

Easy

**Task:** Check if a string is a palindrome after converting uppercase to lowercase and removing non-alphanumeric chars.

```
#include <ctype.h>
#include <stdbool.h>
bool isPalindrome(char* s) {
    int l = 0, r = strlen(s) - 1;
    while (l < r) {
        while (l < r && !isalnum(s[l])) l++;
        while (l < r && !isalnum(s[r])) r--;
        if (tolower(s[l]) != tolower(s[r])) return false;
        l++; r--;
    }
    return true;
}
```

**Core Concept:** Bidirectional pointer convergence while skipping non-alphanumeric characters with `isalnum()`.

### Problem 5: Valid Anagram (LeetCode 242)

Easy

**Task:** Determine if string `t` is an anagram of string `s` (same character frequencies).

```
bool isAnagram(char* s, char* t) {
    int freq[26] = {0};
    for (int i = 0; s[i] != '\0'; i++) freq[s[i] - 'a']++;
    for (int i = 0; t[i] != '\0'; i++) freq[t[i] - 'a']--;
    for (int i = 0; i < 26; i++) {
        if (freq[i] != 0) return false;
    }
    return true;
}
```

**Core Concept:** Frequency counting using a fixed-size 26-element integer hash table. Runs in  $O(N)$  time and  $O(1)$  space.

### Problem 6: First Unique Character in a String (LeetCode 387)

Easy

**Task:** Find the first non-repeating character in a string and return its index. If it does not exist, return -1.

```
int firstUniqChar(char* s) {
    int count[26] = {0};
    for (int i = 0; s[i] != '\0'; i++) count[s[i] - 'a']++;
    for (int i = 0; s[i] != '\0'; i++) {
        if (count[s[i] - 'a'] == 1) return i;
    }
    return -1;
}
```

**Core Concept:** Two-pass linear algorithm. Pass 1 counts frequencies; Pass 2 identifies the first character with a count of 1.

## 5. String Patterns & Manipulations (LeetCode Easy → Medium)

### Problem 7: Find the Index of the First Occurrence (LeetCode 28)

Easy

**Task:** Return index of first occurrence of `needle` in `haystack`, or -1 if not part of it.

```
int strStr(char* haystack, char* needle) {
    int n = strlen(haystack), m = strlen(needle);
    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && haystack[i + j] == needle[j]) j++;
        if (j == m) return i;
    }
    return -1;
}
```

**Core Concept:** Substring matching logic. Simulates standard `strstr()` via nested window checking.

### Problem 8: Longest Common Prefix (LeetCode 14)

Easy

**Task:** Find the longest common prefix string among an array of strings. If none, return `""`.

```
char* longestCommonPrefix(char** strs, int strSize) {
    if (strSize == 0) return "";
    for (int col = 0; strs[0][col] != '\0'; col++) {
        char c = strs[0][col];
        for (int row = 1; row < strSize; row++) {
            if (strs[row][col] != c) {
                strs[0][col] = '\0'; // Terminate in-place
                return strs[0];
            }
        }
    }
    return strs[0];
}
```

**Core Concept:** Vertical character-by-character scanning across string arrays ( `char**` pointers).

### Problem 9: Valid Parentheses (LeetCode 20)

Easy

**Task:** Determine if input bracket sequence containing '(', ')', '{', '}', '[' and ']' is valid.

```
bool isValid(char* s) {
    int n = strlen(s);
    char stack[n + 1];
    int top = -1;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '{' || c == '[') stack[++top] = c;
        else {
            if (top == -1) return false;
            char open = stack[top--];
            if ((c == ')' && open != '(') || (c == '}' && open != '{') ||
                (c == ']' && open != '[')) return false;
        }
    }
    return top == -1;
}
```

**Core Concept:** Array-based Stack simulation in C for matching bracket pairs.

## 6. Intermediate Algorithms (LeetCode Medium) – Part I

### Problem 10: Reverse Words in a String (LeetCode 151)

Medium

**Task:** Reverse order of words in string, trimming multiple spaces into a single space.

```
void reverseRange(char* s, int i, int j) {
    while (i < j) { char t = s[i]; s[i++] = s[j]; s[j--] = t; }
}
char* reverseWords(char* s) {
    int n = strlen(s), w = 0, r = 0;
    while (r < n) {
        while (r < n && s[r] == ' ') r++;
        if (r >= n) break;
        if (w > 0) s[w++] = ' ';
        int start = w;
        while (r < n && s[r] != ' ') s[w++] = s[r++];
        reverseRange(s, start, w - 1);
    }
    s[w] = '\0';
    reverseRange(s, 0, w - 1);
    return s;
}
```

**Core Concept:** Three-step in-place reversal: clean spaces → reverse individual words → reverse entire string.

### Problem 11: String to Integer - atoi (LeetCode 8)

Medium

**Task:** Parse integer from string handling leading spaces, signs, and 32-bit overflow.

```
#include <limits.h>
int myAtoi(char* s) {
    int i = 0, sign = 1; long total = 0;
    while (s[i] == ' ') i++;
    if (s[i] == '+' || s[i] == '-') { sign = (s[i] == '-') ? -1 : 1; i++; }
    while (s[i] >= '0' && s[i] <= '9') {
        total = total * 10 + (s[i] - '0');
        if (sign == 1 && total > INT_MAX) return INT_MAX;
        if (sign == -1 && -total < INT_MIN) return INT_MIN;
        i++;
    }
    return (int)(total * sign);
}
```

**Core Concept:** State parsing with integer overflow clamps ( `INT_MAX` / `INT_MIN` ).

### Problem 12: Longest Palindromic Substring (LeetCode 5)

Medium

**Task:** Given string `s`, return the longest palindromic substring in `s`.

```
int expandAroundCenter(char* s, int l, int r, int len) {
    while (l >= 0 && r < len && s[l] == s[r]) { l--; r++; }
    return r - l - 1;
}
char* longestPalindrome(char* s) {
    int len = strlen(s), start = 0, maxL = 0;
    for (int i = 0; i < len; i++) {
        int l1 = expandAroundCenter(s, i, i, len);
        int l2 = expandAroundCenter(s, i, i + 1, len);
        int cur = l1 > l2 ? l1 : l2;
        if (cur > maxL) { maxL = cur; start = i - (cur - 1) / 2; }
    }
    char* res = (char*)malloc(maxL + 1);
    strncpy(res, s + start, maxL); res[maxL] = '\0';
    return res;
}
```

**Core Concept:** Expand around  $2N - 1$  potential centers (odd and even palindromes) in  $O(N^2)$  time and  $O(1)$  space.

## 7. Sliding Window & Advanced Traversal (LeetCode Medium)

### Problem 13: Longest Substring Without Repeating Characters (LeetCode 3)

Medium

**Task:** Find the length of the longest substring without repeating characters.

```
int lengthOfLongestSubstring(char* s) {
    int lastPos[256];
    for (int i = 0; i < 256; i++) lastPos[i] = -1;
    int maxLen = 0, start = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        unsigned char ch = (unsigned char)s[i];
        if (lastPos[ch] >= start) start = lastPos[ch] + 1;
        lastPos[ch] = i;
        int curLen = i - start + 1;
        if (curLen > maxLen) maxLen = curLen;
    }
    return maxLen;
}
```

**Core Concept:** Sliding window with direct index lookup table (size 256 for all ASCII characters).

### Problem 14: String Compression (LeetCode 443)

Medium

**Task:** Compress character array in-place using count of repeated characters.

```
int compress(char* chars, int charsSize) {
    int write = 0, read = 0;
    while (read < charsSize) {
        char ch = chars[read]; int count = 0;
        while (read < charsSize && chars[read] == ch) { read++; count++; }
        chars[write++] = ch;
        if (count > 1) {
            char buf[12]; int len = sprintf(buf, "%d", count);
            for (int k = 0; k < len; k++) chars[write++] = buf[k];
        }
    }
    return write;
}
```

**Core Concept:** Read/Write two-pointer mechanics with in-place ASCII digit writing using `sprintf`.

### Problem 15: Palindromic Substrings (LeetCode 647)

Medium

**Task:** Count total number of palindromic substrings contained within a string.

```
int countSubstrings(char* s) {
    int n = strlen(s), ans = 0;
    for (int center = 0; center < 2 * n - 1; center++) {
        int l = center / 2;
        int r = l + (center % 2);
        while (l >= 0 && r < n && s[l] == s[r]) { ans++; l--; r++; }
    }
    return ans;
}
```

**Core Concept:** Centered expansion counting both odd and even symmetry pivots across  $O(N)$  points.

#### Freshman Engineering Practice Tip:

Compile using `gcc -Wall -Wextra -Werror program.c`. Always verify that arrays are bounded and allocated with room for the null character `'\0'`.